

DREAM MORE TRANSPORT MANAGEMENT SYSTEM

**(Ride-Sharing and Smart Transport
Platform)**

1. Architecture Overview	3
2. Technology Stack	3
3. Components	3
4. Database Design	3
5. ER Diagram	3
6. Use Case Diagram	3
7. Sequence Diagram	3
8. Class Diagram	3
9. Security Design	3
10. Scalability Design	3

1. Architecture Overview

The system follows a Microservices-ready Monolithic Architecture (Modular Monolith) hosted on a cloud environment. This ensures ease of development while allowing specific components to scale independently.

- Client Layer: Cross-platform mobile apps (Passenger/Driver) and a Web Dashboard (Admin) built with Next.js.
- API Gateway Layer: Handles routing, rate limiting, and authentication.
- Application Logic Layer: Next.js Serverless Functions or a Node.js runtime managing ride-matching algorithms, fare calculations, and geospatial processing.
- Data Layer:
 - PostgreSQL (Primary): Relational data (Users, Rides, Payments)
 - PostGIS (Spatial): Handles geographical queries (finding drivers within X radius).
 - Redis (Cache/Real-time): Stores volatile data like live drive coordinates and active session tokens.
- Integration Layer: External APIs for Payments (Telebirr/Card), SMS Gateways, and Cloudinary for document storage.

2. Technology Stack

Component	Technology	Reason
Frontend	Next.js (React)	Server-side rendering for SEO (landing pages) and fast client-side transitions for dashboards.
Styling	Tailwind CSS	Rapid UI development and responsive design.
Backend	Node.js / Next.js API Routes	Event-driven architecture suitable for real-time updates.
Database	PostgreSQL + PostGIS	Industry standard for relational data with powerful

		spatial indexing for GPS coordinates.
Caching/Pub-Sub	Redis	High-speed read/writes for real-time driver tracking and reducing DB load.
File storage	Cloudinary	Optimized hosting for driver licenses, vehicle photos, and profile images.
Maps/GIS	Google Maps API / Mapbox	Route optimization, geocoding, and visual map rendering.
Auth	NextAuth.js / JWT	Secure session management and Role-Based Access Control (RBAC).

3. Components

1. Identity Service: Handles registration, login, and KYC (Know Your Customer) verification for drivers.
2. Geospatial Matching Engine: The "Brain" of the system. It uses PostGIS to find the nearest available drivers to a passenger's pickup point.
3. Fare & Surge Pricing Engine: Calculates costs based on distance, time, and demand-supply ratios (Surge).
4. Real-Time Dispatcher: Uses WebSockets (Socket.io) or Redis Pub/Sub to push ride requests to drivers in real-time.
5. Payment Gateway Bridge: An abstraction layer to connect with Telebirr, eBirr, and international card processors.
6. Notification Service: Manages Push Notifications (Firebase Cloud Messaging), SMS, and Emails.

4. Database Design

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

```
CREATE TABLE Users (  
  user_id      SERIAL PRIMARY KEY,  
  full_name    VARCHAR(100) NOT NULL,  
  email        VARCHAR(100) UNIQUE NOT NULL,  
  phone_number VARCHAR(20) UNIQUE NOT NULL,  
  password_hash TEXT NOT NULL,  
  role         VARCHAR(20) NOT NULL CHECK (role IN ('passenger', 'driver', 'admin')),  
  profile_photo TEXT,  
  is_active    BOOLEAN DEFAULT TRUE,  
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE Vehicles (  
  vehicle_id    SERIAL PRIMARY KEY,  
  plate_number  VARCHAR(20) UNIQUE NOT NULL,  
  model         VARCHAR(100) NOT NULL,  
  manufacturer  VARCHAR(100) NOT NULL,  
  year         INTEGER NOT NULL,  
  color         VARCHAR(50) NOT NULL,  
  vehicle_tier  VARCHAR(20) NOT NULL CHECK (vehicle_tier IN ('economy', 'comfort',  
'premium')),  
  seat_capacity INTEGER NOT NULL,  
  is_electric   BOOLEAN DEFAULT FALSE,  
  is_active     BOOLEAN DEFAULT TRUE,  
  created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE Drivers (  
  driver_id    SERIAL PRIMARY KEY,  
  user_id      INTEGER UNIQUE NOT NULL REFERENCES Users(user_id) ON DELETE  
CASCADE,  
  vehicle_id   INTEGER UNIQUE NOT NULL REFERENCES Vehicles(vehicle_id) ON  
DELETE RESTRICT,  
  licence_number VARCHAR(50) UNIQUE NOT NULL,  
  licence_expiry DATE NOT NULL,  
  is_verified   BOOLEAN DEFAULT FALSE,  
  is_available  BOOLEAN DEFAULT FALSE,  
  current_latitude DECIMAL(9, 6),  
  current_longitude DECIMAL(9, 6),  
  rating        DECIMAL(3, 2) DEFAULT 0.00,
```

```

    total_rides    INTEGER DEFAULT 0,
    created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```

CREATE TABLE Driver_Documents (
    document_id    SERIAL PRIMARY KEY,
    driver_id      INTEGER NOT NULL REFERENCES Drivers(driver_id) ON DELETE
CASCADE,
    document_type  VARCHAR(50) NOT NULL CHECK (document_type IN (
        'licence', 'vehicle_registration', 'insurance'
    )),
    file_url       TEXT NOT NULL,
    verified       BOOLEAN DEFAULT FALSE,
    uploaded_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```

CREATE TABLE Rides (
    ride_id        SERIAL PRIMARY KEY,
    driver_id      INTEGER REFERENCES Drivers(driver_id) ON DELETE SET NULL,
    pickup_latitude DECIMAL(9, 6) NOT NULL,
    pickup_longitude DECIMAL(9, 6) NOT NULL,
    pickup_landmark VARCHAR(255),
    dropoff_latitude DECIMAL(9, 6) NOT NULL,
    dropoff_longitude DECIMAL(9, 6) NOT NULL,
    dropoff_landmark VARCHAR(255),
    ride_type      VARCHAR(20) NOT NULL CHECK (ride_type IN ('shared', 'private')),
    fare_type      VARCHAR(20) NOT NULL CHECK (fare_type IN ('fixed', 'negotiated')),
    base_fare      DECIMAL(10, 2) NOT NULL,
    final_fare     DECIMAL(10, 2),
    distance_km    DECIMAL(6, 2),
    duration_minutes INTEGER,
    surge_multiplier DECIMAL(4, 2) DEFAULT 1.00,
    discount_amount DECIMAL(10, 2) DEFAULT 0.00,
    status         VARCHAR(20) NOT NULL CHECK (status IN (
        'pending', 'accepted', 'in_progress',
        'completed', 'cancelled'
    )),
    scheduled_at   TIMESTAMP,
    started_at     TIMESTAMP,
    completed_at   TIMESTAMP,
    cancelled_at   TIMESTAMP,
    cancellation_reason TEXT,
    cancelled_by   VARCHAR(20) CHECK (cancelled_by IN ('passenger', 'driver', 'system')),
    created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

);

```
CREATE TABLE Ride_Passengers (  
  ride_passenger_id SERIAL PRIMARY KEY,  
  ride_id INTEGER NOT NULL REFERENCES Rides(ride_id) ON DELETE CASCADE,  
  passenger_id INTEGER NOT NULL REFERENCES Users(user_id) ON DELETE  
CASCADE,  
  fare_share DECIMAL(10, 2) NOT NULL,  
  payment_status VARCHAR(20) NOT NULL DEFAULT 'pending' CHECK (payment_status  
IN (  
    'pending', 'paid', 'failed', 'refunded'  
  )),  
  pickup_latitude DECIMAL(9, 6),  
  pickup_longitude DECIMAL(9, 6),  
  pickup_landmark VARCHAR(255),  
  dropoff_latitude DECIMAL(9, 6),  
  dropoff_longitude DECIMAL(9, 6),  
  dropoff_landmark VARCHAR(255),  
  booked_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  joined_at TIMESTAMP,  
  dropped_off_at TIMESTAMP,  
  UNIQUE (ride_id, passenger_id)  
);
```

```
CREATE TABLE Payments (  
  payment_id SERIAL PRIMARY KEY,  
  ride_passenger_id INTEGER NOT NULL REFERENCES  
Ride_Passengers(ride_passenger_id) ON DELETE RESTRICT,  
  amount DECIMAL(10, 2) NOT NULL,  
  payment_method VARCHAR(20) NOT NULL CHECK (payment_method IN (  
    'cash', 'card', 'telebirr', 'ebirr', 'wallet'  
  )),  
  payment_status VARCHAR(20) NOT NULL DEFAULT 'pending' CHECK (payment_status  
IN (  
    'pending', 'completed', 'failed', 'refunded'  
  )),  
  transaction_ref VARCHAR(255) UNIQUE,  
  initiated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  completed_at TIMESTAMP,  
  refunded_at TIMESTAMP,  
  refund_reason TEXT  
);
```

```
CREATE TABLE Ratings (  

```

```

    rating_id    SERIAL PRIMARY KEY,
    ride_id      INTEGER NOT NULL REFERENCES Rides(ride_id) ON DELETE CASCADE,
    rated_by     INTEGER NOT NULL REFERENCES Users(user_id) ON DELETE
CASCADE,
    rated_user   INTEGER NOT NULL REFERENCES Users(user_id) ON DELETE
CASCADE,
    score        INTEGER NOT NULL CHECK (score BETWEEN 1 AND 5),
    comment      TEXT,
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (ride_id, rated_by, rated_user)
);

```

```

CREATE TABLE Notifications (
    notification_id SERIAL PRIMARY KEY,
    user_id         INTEGER NOT NULL REFERENCES Users(user_id) ON DELETE
CASCADE,
    title           VARCHAR(255) NOT NULL,
    message         TEXT NOT NULL,
    type            VARCHAR(50) NOT NULL CHECK (type IN (
        'ride_accepted', 'driver_arrival', 'ride_started',
        'ride_completed', 'payment_confirmed', 'ride_cancelled'
    )),
    is_read         BOOLEAN DEFAULT FALSE,
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```

CREATE TABLE SOS_Alerts (
    alert_id       SERIAL PRIMARY KEY,
    ride_id        INTEGER REFERENCES Rides(ride_id) ON DELETE SET NULL,
    triggered_by   INTEGER NOT NULL REFERENCES Users(user_id) ON DELETE
CASCADE,
    latitude       DECIMAL(9, 6),
    longitude      DECIMAL(9, 6),
    status         VARCHAR(20) DEFAULT 'triggered' CHECK (status IN (
        'triggered', 'responded', 'resolved'
    )),
    triggered_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    resolved_at    TIMESTAMP
);

```

```

CREATE TABLE Complaints (
    complaint_id   SERIAL PRIMARY KEY,
    ride_id        INTEGER REFERENCES Rides(ride_id) ON DELETE SET NULL,

```



```

        submitted_by    INTEGER NOT NULL REFERENCES Users(user_id) ON DELETE
CASCADE,
        complaint_against INTEGER REFERENCES Users(user_id) ON DELETE SET NULL,
        description      TEXT NOT NULL,
        status           VARCHAR(20) DEFAULT 'open' CHECK (status IN (
                                'open', 'under_review', 'resolved', 'dismissed'
                                )),
        resolution       TEXT,
        created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        resolved_at      TIMESTAMP
    );

CREATE INDEX idx_drivers_location ON Drivers USING GIST (
    ST_MakePoint(current_longitude, current_latitude)
);

CREATE INDEX idx_rides_pickup ON Rides USING GIST (
    ST_MakePoint(pickup_longitude, pickup_latitude)
);

CREATE INDEX idx_rides_dropoff ON Rides USING GIST (
    ST_MakePoint(dropoff_longitude, dropoff_latitude)
);

```

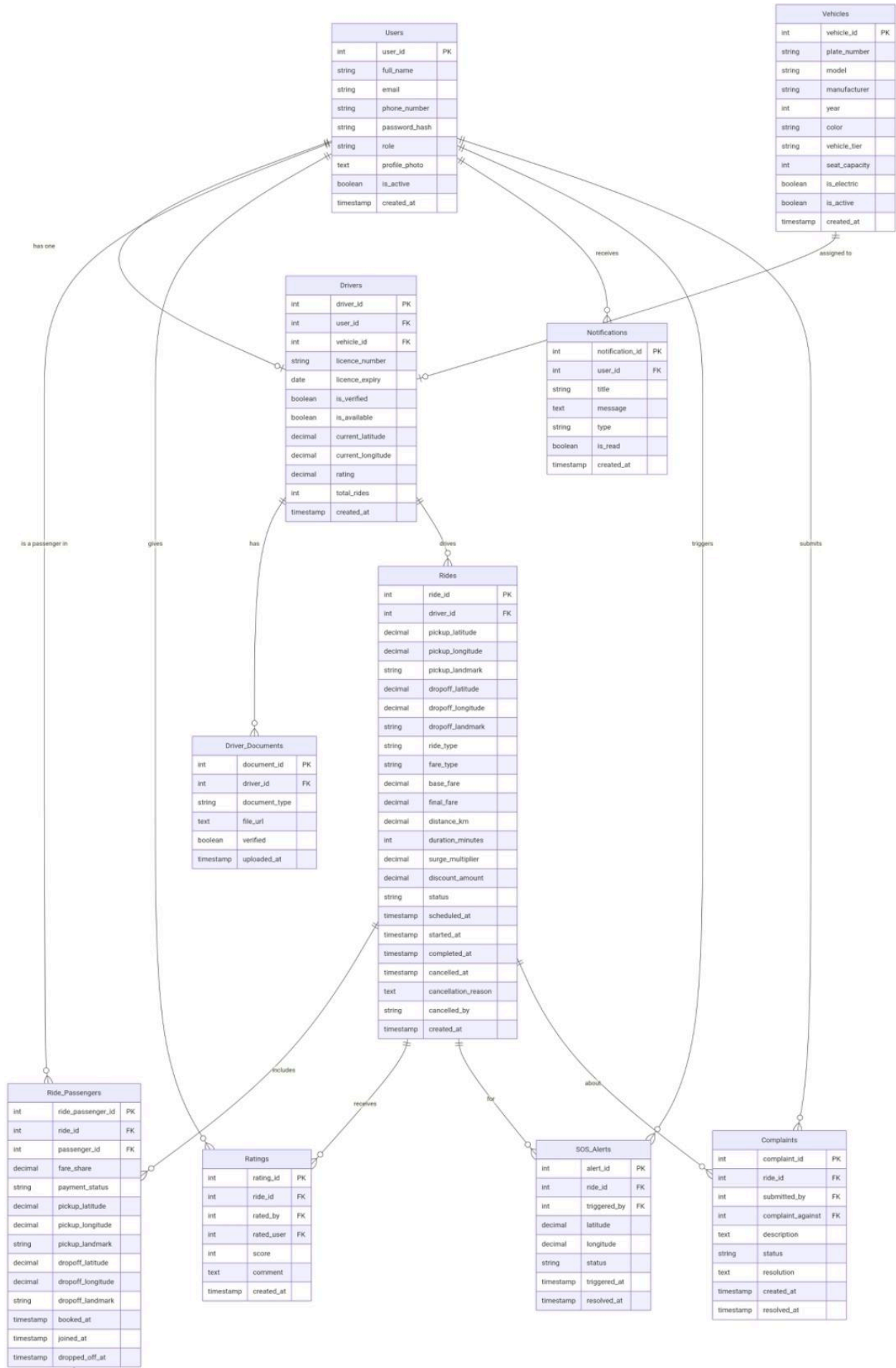
5. ER Diagram

Based on the provided SRS and SADD documents, here are the UML diagrams for the Dream More Transport Management System, designed to incorporate the modern features you've outlined.

1. ER Diagram (Entity-Relationship Diagram)

This diagram visualizes the database schema, highlighting the key entities and their relationships, including the modern feature of Ride Sharing and Cost Splitting.

The ER diagram is here



The code of the ER diagram

```
```mermaid
erDiagram
 Users {
 int user_id PK
 string full_name
 string email
 string phone_number
 string password_hash
 string role
 text profile_photo
 boolean is_active
 timestamp created_at
 }

 Drivers {
 int driver_id PK
 int user_id FK
 int vehicle_id FK
 string licence_number
 date licence_expiry
 boolean is_verified
 boolean is_available
 decimal current_latitude
 decimal current_longitude
 decimal rating
 int total_rides
 timestamp created_at
 }

 Vehicles {
 int vehicle_id PK
 string plate_number
 string model
 string manufacturer
 int year
 string color
 string vehicle_tier
 int seat_capacity
 boolean is_electric
 boolean is_active
 timestamp created_at
 }
```

```
}
```

```
Driver_Documents {
 int document_id PK
 int driver_id FK
 string document_type
 text file_url
 boolean verified
 timestamp uploaded_at
}
```

```
Rides {
 int ride_id PK
 int driver_id FK
 decimal pickup_latitude
 decimal pickup_longitude
 string pickup_landmark
 decimal dropoff_latitude
 decimal dropoff_longitude
 string dropoff_landmark
 string ride_type
 string fare_type
 decimal base_fare
 decimal final_fare
 decimal distance_km
 int duration_minutes
 decimal surge_multiplier
 decimal discount_amount
 string status
 timestamp scheduled_at
 timestamp started_at
 timestamp completed_at
 timestamp cancelled_at
 text cancellation_reason
 string cancelled_by
 timestamp created_at
}
```

```
Ride_Passengers {
 int ride_passenger_id PK
 int ride_id FK
 int passenger_id FK
 decimal fare_share
 string payment_status
```

```
 decimal pickup_latitude
 decimal pickup_longitude
 string pickup_landmark
 decimal dropoff_latitude
 decimal dropoff_longitude
 string dropoff_landmark
 timestamp booked_at
 timestamp joined_at
 timestamp dropped_off_at
}
```

```
Payments {
 int payment_id PK
 int ride_passenger_id FK
 decimal amount
 string payment_method
 string payment_status
 string transaction_ref
 timestamp initiated_at
 timestamp completed_at
 timestamp refunded_at
 text refund_reason
}
```

```
Ratings {
 int rating_id PK
 int ride_id FK
 int rated_by FK
 int rated_user FK
 int score
 text comment
 timestamp created_at
}
```

```
Notifications {
 int notification_id PK
 int user_id FK
 string title
 text message
 string type
 boolean is_read
 timestamp created_at
}
```

```
SOS_Alerts {
 int alert_id PK
 int ride_id FK
 int triggered_by FK
 decimal latitude
 decimal longitude
 string status
 timestamp triggered_at
 timestamp resolved_at
}
```

```
Complaints {
 int complaint_id PK
 int ride_id FK
 int submitted_by FK
 int complaint_against FK
 text description
 string status
 text resolution
 timestamp created_at
 timestamp resolved_at
}
```

```
Users ||--o| Drivers : "has one"
Users ||--o{ Ride_Passengers : "is a passenger in"
Users ||--o{ Ratings : "gives"
Users ||--o{ Notifications : "receives"
Users ||--o{ SOS_Alerts : "triggers"
Users ||--o{ Complaints : "submits"
```

```
Vehicles ||--o| Drivers : "assigned to"
Drivers ||--o{ Driver_Documents : "has"
Drivers ||--o{ Rides : "drives"
```

```
Rides ||--o{ Ride_Passengers : "includes"
Rides ||--o{ Ratings : "receives"
Rides ||--o{ SOS_Alerts : "for"
Rides ||--o{ Complaints : "about"
```

```
Ride_Passengers ||--o{ Payments : "has"
```

...

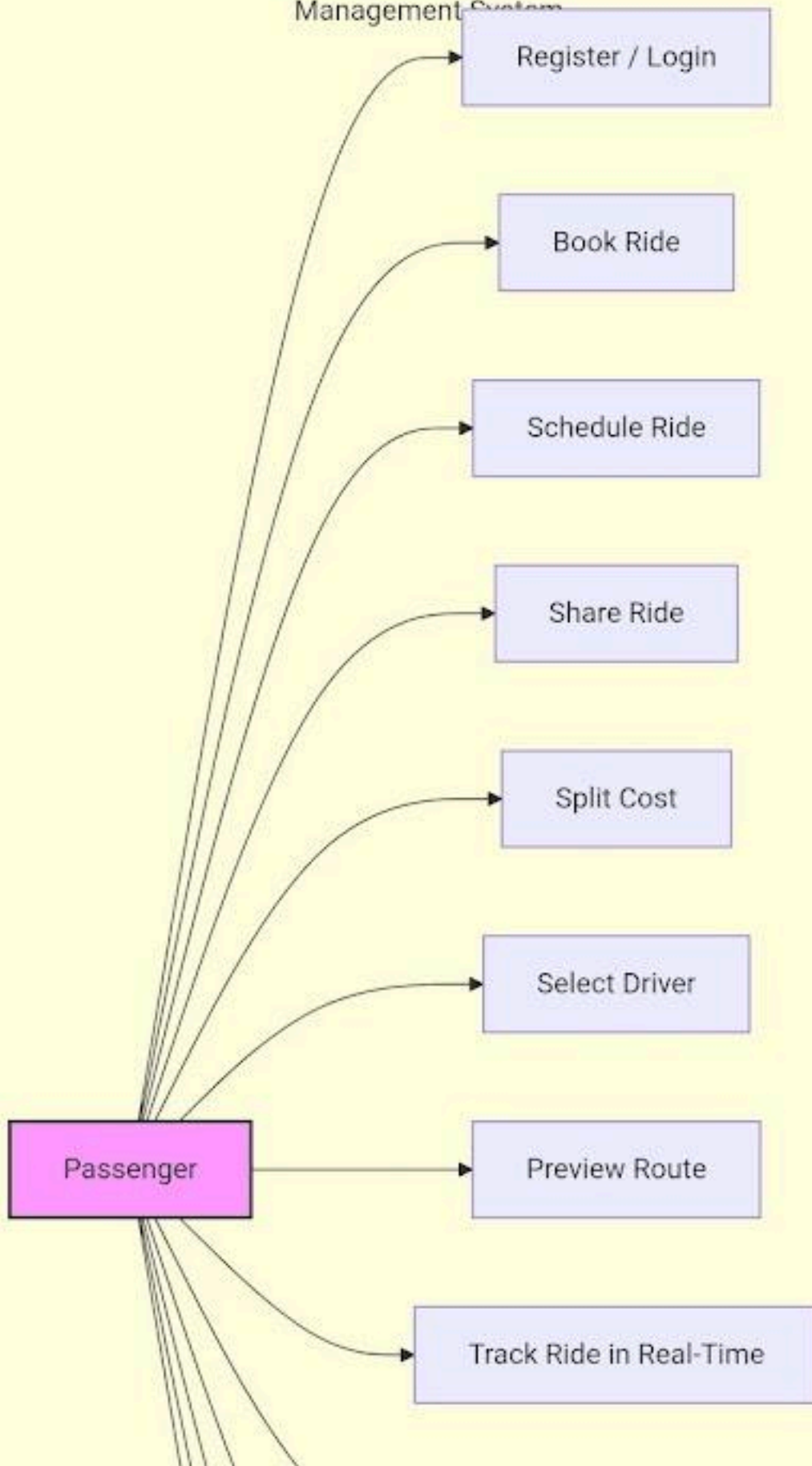
---

## 6. Use Case Diagram

This diagram outlines the high-level interactions for the three main actors: Passenger, Driver, and Admin. It highlights the modern features like Passenger Selects Driver, Cost Splitting, and Route Preview.

The diagram is in the bellow

Dream More Transport  
Management System





Use case diagram code

```
```mermaid
```

```
graph TD
```

```
    subgraph "Dream More Transport Management System"
```

```
        A[Passenger] --> B[Register / Login]
```

```
        A --> C[Book Ride]
```

```
        A --> D[Schedule Ride]
```

```
        A --> E[Share Ride]
```

```
        A --> F[Split Cost]
```

```
        A --> G[Select Driver]
```

```
        A --> H[Preview Route]
```

```
        A --> I[Track Ride in Real-Time]
```

```
        A --> J[Make Payment]
```

```
        A --> K[Rate Driver]
```

```
        A --> L[View Ride History]
```

```
        A --> M[Trigger SOS Alert]
```

```
        A --> N[Submit Complaint]
```

```
        O[Driver] --> P[Register / Login]
```

```
        O --> Q[Manage Availability]
```

```
        O --> R[Receive Ride Requests]
```

```
        O --> S[Accept/Reject Ride]
```

```
        O --> T[View Full Route Before Acceptance]
```

```
        O --> U[Navigate to Pickup/Drop-off]
```

```
        O --> V[Update Ride Status]
```

```
        O --> W[View Earnings]
```

```
        O --> X[Rate Passenger]
```

```
        Y[Admin] --> Z[Manage Users]
```

```
        Y --> AA[Verify Drivers]
```

```
        Y --> AB[Monitor Rides]
```

```
        Y --> AC[Manage Payments/Disputes]
```

```
        Y --> AD[Generate Reports]
```

```
        Y --> AE[Manage Complaints]
```

```
    end
```

```
    style A fill:#f9f,stroke:#333,stroke-width:2px
```

```
    style O fill:#ccf,stroke:#333,stroke-width:2px
```

```
    style Y fill:#cfc,stroke:#333,stroke-width:2px
```

```
```
```

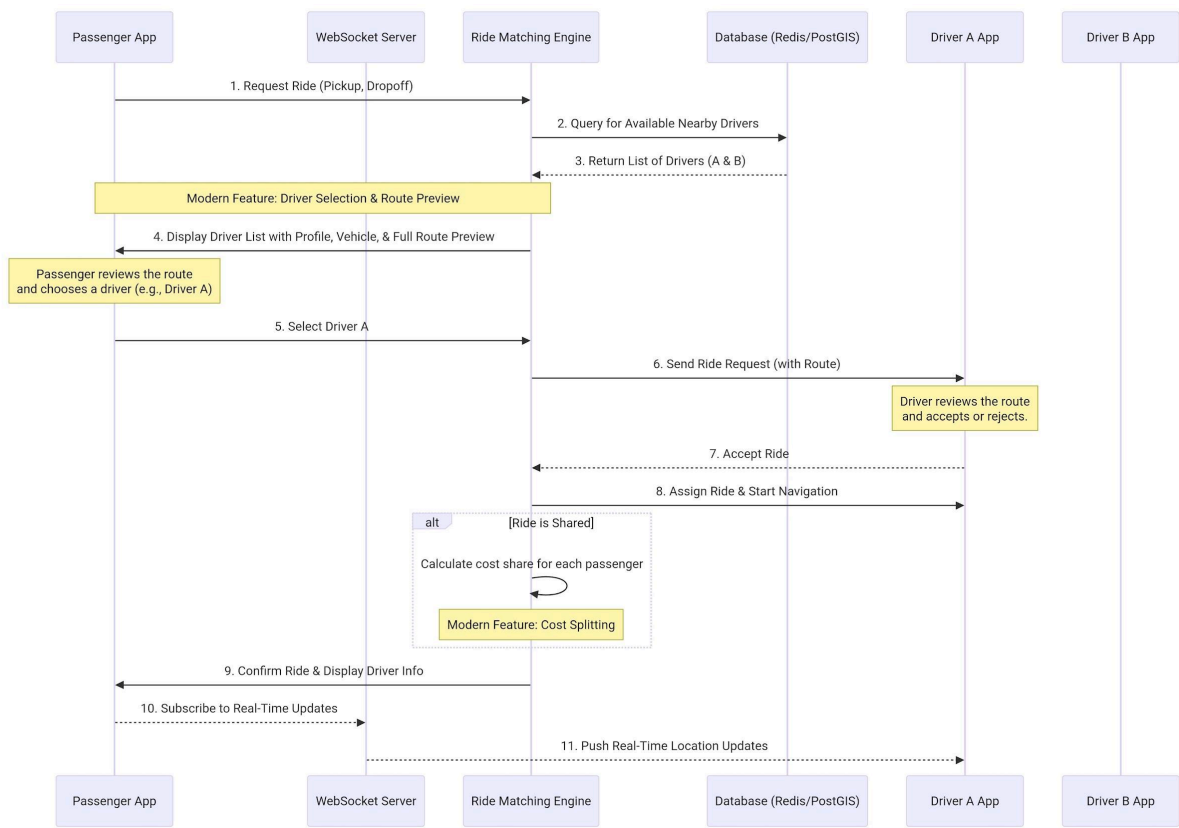
—

# 7. Sequence Diagram

Sequence Diagram (for "Book a Ride" with Modern Features)

This sequence diagram illustrates the flow of a passenger booking a ride, incorporating the modern features: Route Preview before Acceptance and Passenger Selects Driver.

The diagram of sequence



The sequence diagram code

```
```mermaid
```

sequenceDiagram

participant Passenger App

participant WebSocket Server

participant Ride Matching Engine

participant Database (Redis/PostGIS)

participant Driver A App

participant Driver B App

Passenger App->>Ride Matching Engine: 1. Request Ride (Pickup, Dropoff)

Ride Matching Engine->>Database (Redis/PostGIS): 2. Query for Available Nearby Drivers
Database (Redis/PostGIS)-->>Ride Matching Engine: 3. Return List of Drivers (A & B)

Note over Ride Matching Engine, Passenger App: Modern Feature: Driver Selection & Route Preview

Ride Matching Engine->>Passenger App: 4. Display Driver List with Profile, Vehicle, & Full Route Preview

Note over Passenger App: Passenger reviews the route
and chooses a driver (e.g., Driver A)

Passenger App->>Ride Matching Engine: 5. Select Driver A

Ride Matching Engine->>Driver A App: 6. Send Ride Request (with Route)

Note over Driver A App: Driver reviews the route
and accepts or rejects.

Driver A App-->>Ride Matching Engine: 7. Accept Ride

Ride Matching Engine->>Driver A App: 8. Assign Ride & Start Navigation

alt Ride is Shared

Ride Matching Engine->>Ride Matching Engine: Calculate cost share for each passenger

Note over Ride Matching Engine: Modern Feature: Cost Splitting
end

Ride Matching Engine->>Passenger App: 9. Confirm Ride & Display Driver Info

Passenger App-->>WebSocket Server: 10. Subscribe to Real-Time Updates

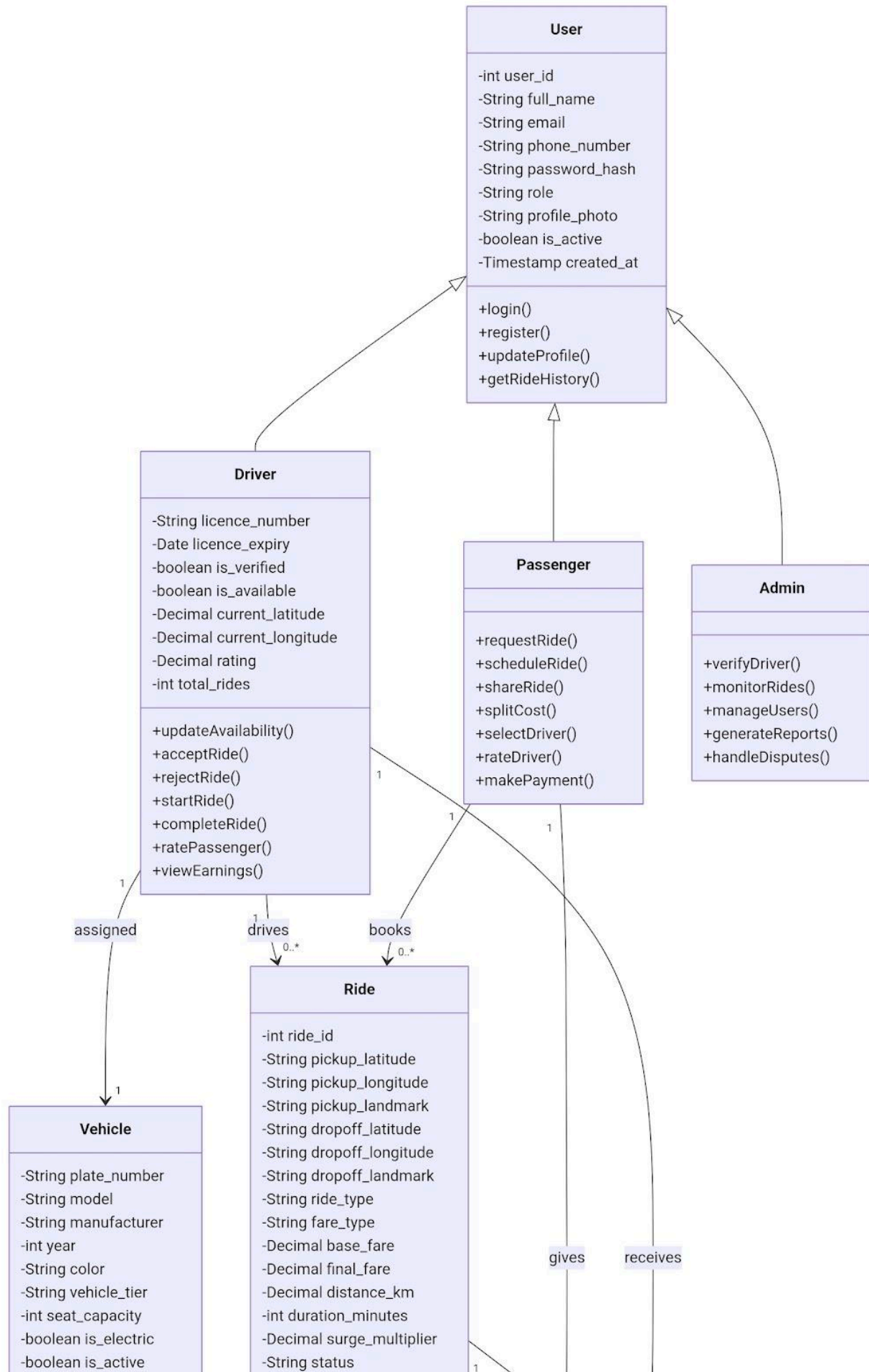
WebSocket Server-->>Driver A App: 11. Push Real-Time Location Updates

...

8. Class Diagram

This diagram shows the static structure of the system's core classes and their relationships, designed to support the modern features of Ride Sharing and Cost Splitting.

Diagram



The code of the diagram

```
```mermaid
classDiagram
 class User {
 -int user_id
 -String full_name
 -String email
 -String phone_number
 -String password_hash
 -String role
 -String profile_photo
 -boolean is_active
 -Timestamp created_at
 +login()
 +register()
 +updateProfile()
 +getRideHistory()
 }

 class Passenger {
 +requestRide()
 +scheduleRide()
 +shareRide()
 +splitCost()
 +selectDriver()
 +rateDriver()
 +makePayment()
 }

 class Driver {
 -String licence_number
 -Date licence_expiry
 -boolean is_verified
 -boolean is_available
 -Decimal current_latitude
 -Decimal current_longitude
 -Decimal rating
 -int total_rides
 +updateAvailability()
 +acceptRide()
 +rejectRide()
 +startRide()
 +completeRide()
 }
```

```
+ratePassenger()
+viewEarnings()
}

class Vehicle {
 -String plate_number
 -String model
 -String manufacturer
 -int year
 -String color
 -String vehicle_tier
 -int seat_capacity
 -boolean is_electric
 -boolean is_active
}

class Ride {
 -int ride_id
 -String pickup_latitude
 -String pickup_longitude
 -String pickup_landmark
 -String dropoff_latitude
 -String dropoff_longitude
 -String dropoff_landmark
 -String ride_type
 -String fare_type
 -Decimal base_fare
 -Decimal final_fare
 -Decimal distance_km
 -int duration_minutes
 -Decimal surge_multiplier
 -String status
 -Timestamp scheduled_at
 +calculateFare()
 +assignDriver()
 +updateStatus()
 +cancelRide()
 +previewRoute()
}

class RidePassenger {
 -int ride_passenger_id
 -Decimal fare_share
 -String payment_status
```

```
+confirmParticipation()
+updatePaymentStatus()
}
```

```
class Payment {
 -int payment_id
 -Decimal amount
 -String payment_method
 -String payment_status
 -String transaction_ref
 +processPayment()
 +refundPayment()
 +splitCost()
}
```

```
class Rating {
 -int rating_id
 -int score
 -String comment
 +submitRating()
}
```

```
class Notification {
 -String title
 -String message
 -String type
 -boolean is_read
 +sendNotification()
}
```

```
class Admin {
 +verifyDriver()
 +monitorRides()
 +manageUsers()
 +generateReports()
 +handleDisputes()
}
```

```
User <|-- Passenger
User <|-- Driver
User <|-- Admin
Driver "1" --> "1" Vehicle : assigned
Driver "1" --> "0..*" Ride : drives
Passenger "1" --> "0..*" Ride : books
```

```

Ride "1" --> "1..*" RidePassenger : includes
RidePassenger "1" --> "1" Payment : has
Ride "1" --> "0..*" Rating : receives
Ride "1" --> "0..*" Notification : triggers
Passenger "1" --> "0..*" Rating : gives
Driver "1" --> "0..*" Rating : receives
...

```

These diagrams provide a comprehensive view of the Dream More Transport Management System, reflecting both the core functionalities outlined in the SRS and the innovative, user-centric features detailed in the SADD.

## 9. Scalability Design

Scalability & Security Design for Dream More Transport Management System



Scalability Design (10k+ Users)

### 1. Horizontal Scaling Strategy

- Stateless Application Servers: Deploy multiple Next.js/Node.js instances behind a load balancer (AWS ELB / Nginx)
- Database Read Replicas: PostgreSQL with read replicas for analytical queries and reporting
- Sharding Strategy: Partition Rides and Payments tables by ride\_id or by time (monthly/yearly partitions)
- Caching Layer: Redis cluster for:
  - Active driver locations (TTL: 5 seconds)
  - Session tokens
  - Frequently accessed user profiles
  - Fare calculation results

### 2. Performance Optimization

...

- PostGIS Spatial Indexes: GiST indexes on location columns
- Connection Pooling: PgBouncer for database connections
- Async Processing: Bull/Redis queues for:
  - \* Email/SMS notifications
  - \* Payment processing
  - \* Document verification



- CDN: Cloudflare/CloudFront for static assets and profile photos

...

### 3. Real-time Scaling

- WebSocket Scaling: Use Redis Pub/Sub to sync WebSocket connections across multiple servers
- Geospatial Matching: Implement cell-based partitioning (Uber's H3 / Google S2) for faster driver discovery

### 4. Auto-scaling Rules

- CPU > 70%: Add instance
- Memory > 80%: Add instance
- Request queue length > 100: Scale up
- Off-peak hours: Scale down to minimum

## 10. Security Design

### Security Design

#### 1. Authentication & Authorization

- JWT (JSON Web Tokens):
  - Short-lived access tokens (15 min expiry)
  - Refresh tokens (7 days) stored in HTTP-only cookies
- Password Security:
  - bcrypt hashing (salt rounds: 12)
  - Rate limiting: 5 attempts/hour per IP
- Role-Based Access Control (RBAC):

...

ADMIN: Full system access

DRIVER: Ride management, profile update

PASSENGER: Booking, payment, ride history

...

#### 2. API Security

...

- HTTPS/TLS 1.3 mandatory
  - API Gateway rate limiting: 100 requests/minute per user
  - CORS: Whitelist trusted domains only
  - Input validation: Joi/Zod schemas
  - SQL Injection prevention: Parameterized queries (Sequelize/Knex)
  - XSS protection: Content-Security-Policy headers
- ...

### 3. Data Protection

- Encryption at Rest: AWS RDS encryption / Azure Disk Encryption
- Encryption in Transit: TLS for all external communications
- Sensitive Data:
  - Payment details tokenized (PCI-DSS compliance)
  - Personal data pseudonymization for analytics
  - Automatic PII masking in logs

### 4. Audit & Monitoring

- Logging: All critical actions logged (login, ride status changes, payments)
- Monitoring:
  - Failed login attempts
  - Unusual ride patterns
  - Suspicious payment activities
- Security Headers:

...

Strict-Transport-Security: max-age=31536000  
 X-Frame-Options: DENY  
 X-Content-Type-Options: nosniff  
 Referrer-Policy: strict-origin-when-cross-origin

...

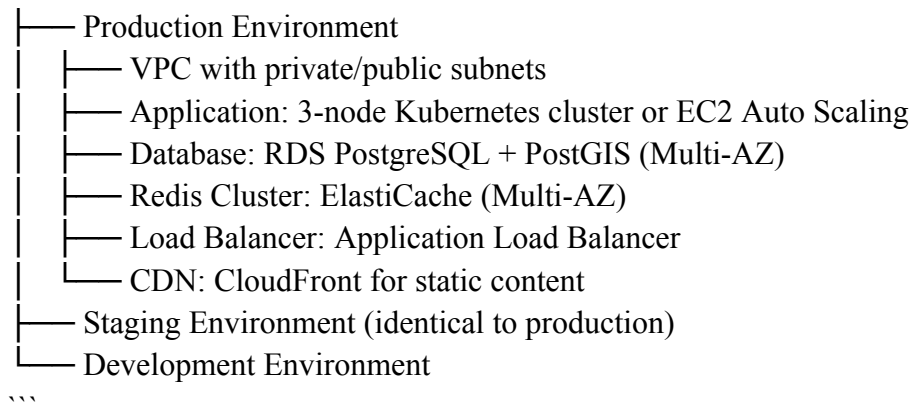
---

## Deployment Plan

### Cloud Infrastructure (Multi-Region)

...

Region: Primary (Ethiopia - AWS/Azure/GCP)



## CI/CD Pipeline

- ...
1. Code Push → GitHub/GitLab
  2. Lint & Test → GitHub Actions/GitLab CI
  3. Build → Docker image (Next.js + Node.js)
  4. Security Scan → Snyk/Trivy
  5. Deploy → Kubernetes rolling update / Blue-Green
  6. Smoke Test → Health endpoints verification
  7. Rollback → Automated on failure
- ...

## Disaster Recovery

- RPO (Recovery Point Objective): 15 minutes (WAL archiving to S3)
- RTO (Recovery Time Objective): 1 hour
- Backup Strategy:
  - Daily full database backup (encrypted)
  - Continuous WAL archiving
  - Multi-region backup replication

## Monitoring Stack

- ...
- Prometheus + Grafana: Metrics & dashboards
  - ELK Stack: Centralized logging
  - Sentry: Error tracking
  - Uptime monitoring: 5-minute interval checks
  - Alerting: Slack/PagerDuty integration

...

## Cost Optimization

- Spot Instances: For non-critical workloads
- Auto-scaling: Scale down during off-peak hours (12 AM - 5 AM)
- Reserved Instances: For database and production servers
- S3 Lifecycle: Move old logs to Glacier after 30 days

---

## Quick Deployment Checklist

- SSL certificates configured
- Security groups/network ACLs locked down
- Database backups configured
- Monitoring & alerting active
- Rollback strategy tested
- Load testing performed
- GDPR/Data privacy compliance reviewed
- Disaster recovery plan documented